



OPERATING SYSTEMS

UE24CS242B

Deadlocks

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course

- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:
 1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
 2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
 3. Some presentation transcripts from A. Frank – P. Weisberg
 4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau



OPERATING SYSTEMS



Principles, Detection, Prevention and Avoidance

Pavan A C

Department of Computer Science & Engineering

- Multiprogramming environment: several processes compete to limited number of resources
- A Process is holding a resource(R1) and is waiting for the resources(R2).
- The resource R2 is held by another process..
- Waiting state of processes will not change, as the requested resource is held by the waiting process.
- This situation is called deadlock

- System consists of finite number of resources
- Resource types $R_1, R_2, \dots, R_m$
 - *Physical resources: CPU cycles, memory space, I/O devices, printer, tape drives.*
 - *Logical resources: semaphores, mutex locks, files.*
- Each resource type R_i has W_i instances.
- Ex: if the system has 2 CPU's then CPU has 2 instances
- Each process utilizes a resource as follows:
 - **Request** : Process makes request to the resource. Eg. system call like request(), open(), wait(), allocate() etc
 - **Use**: operates on these resources
 - **Release**: process releases the resources. Eg. Using a system call like release(), close(), signal(), free etc.
- Request and release of semaphore, acquire and release of lock on mutex

OPERATING SYSTEMS

Deadlock with physical resources

Example 1

- Consider a system with 3 CD RW drives.
- Suppose 3 processes(p_0, p_1, p_2) are holding one drive each.
- What happens,
 - If a process p_0 makes a request for one more drive

Example 2

- Consider a system with one printer one DVD drive.
- Process P_i is holding printer and process P_j is holding DVD drive.
- What happens if
 - Process P_i request DVD and P_j requests printer

Does dead lock occur in example 1 and 2?



- Data:
 - A semaphore **S1** initialized to 1
 - A semaphore **S2** initialized to 1

- Two processes P1 and P2

- **P1:**

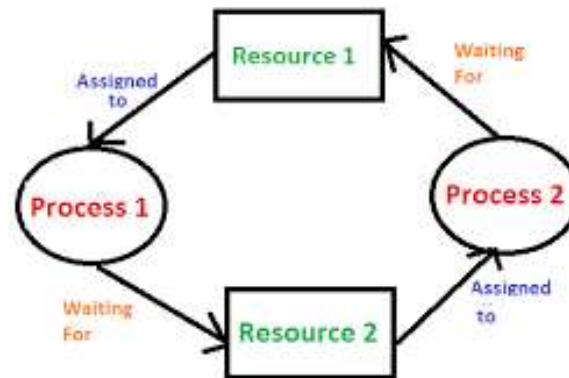
wait(s1)

wait(s2)

- **P2:**

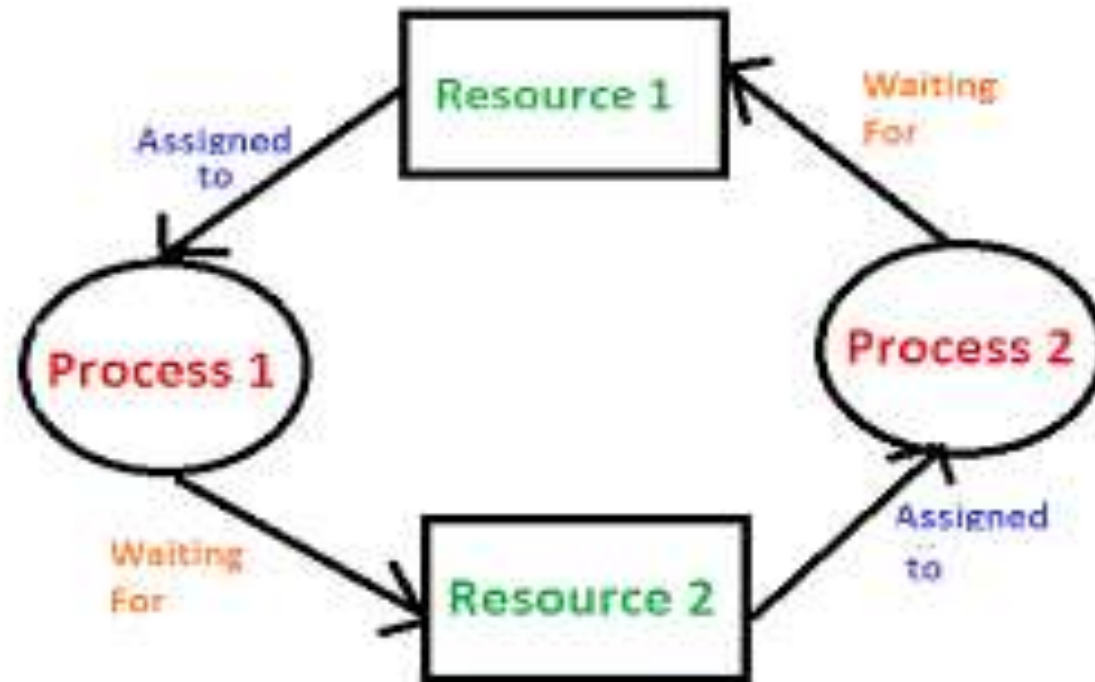
wait(s2)

wait(s1)



OPERATING SYSTEMS

Deadlock with Semaphores



Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource (sharable resources like Read-only files do not require mutually exclusive access and thus cannot be involved in a deadlock).
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

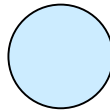
- Deadlocks are described precisely with directed graphs called system resource-allocation graph

A graph consists of set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

A set of vertices V and a set of edges E .

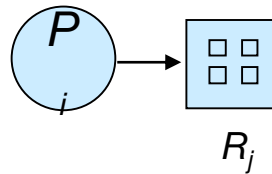
- Process



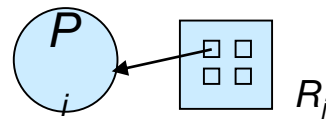
- Resource Type with 4 instances



- P_i requests instance of R_j

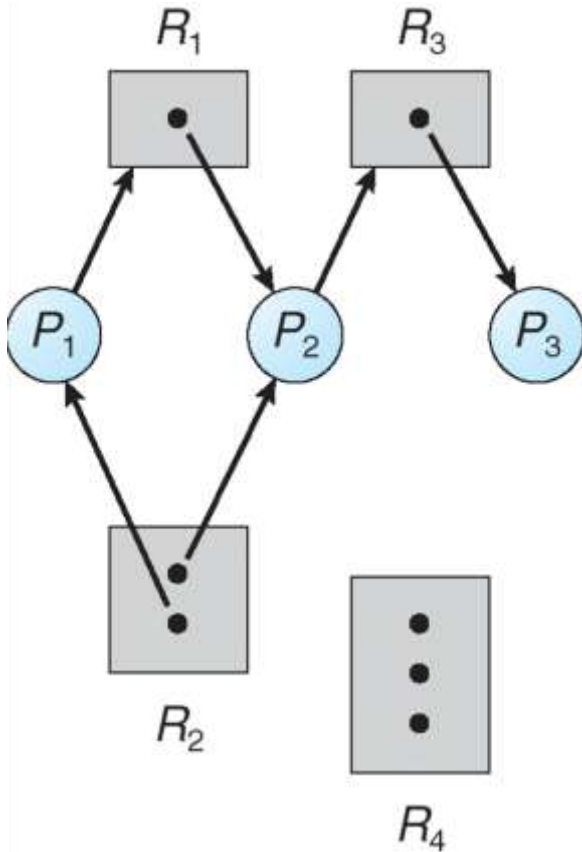


- P_i is holding an instance of R_j



OPERATING SYSTEMS

Example of a Resource-Allocation Graph

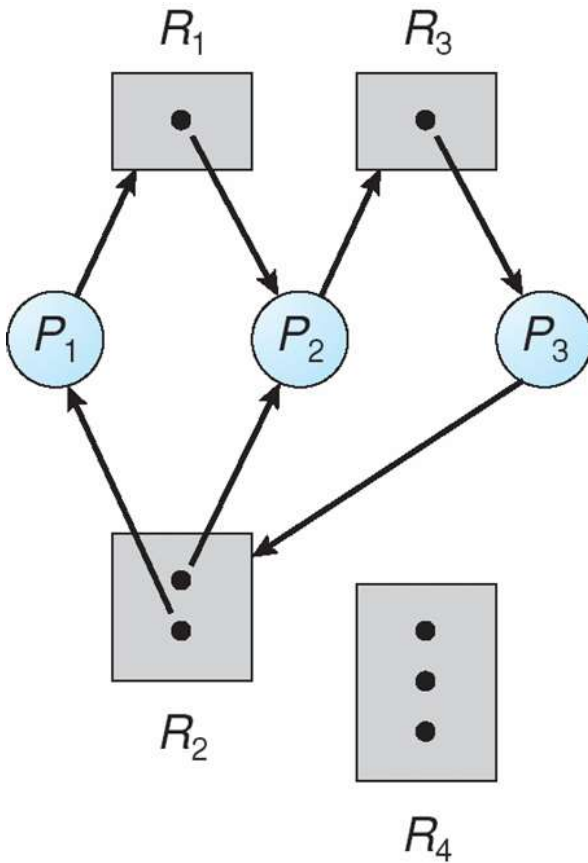


Set of process: P_1, P_2, P_3

Set of Resources: R_1, R_2, R_3

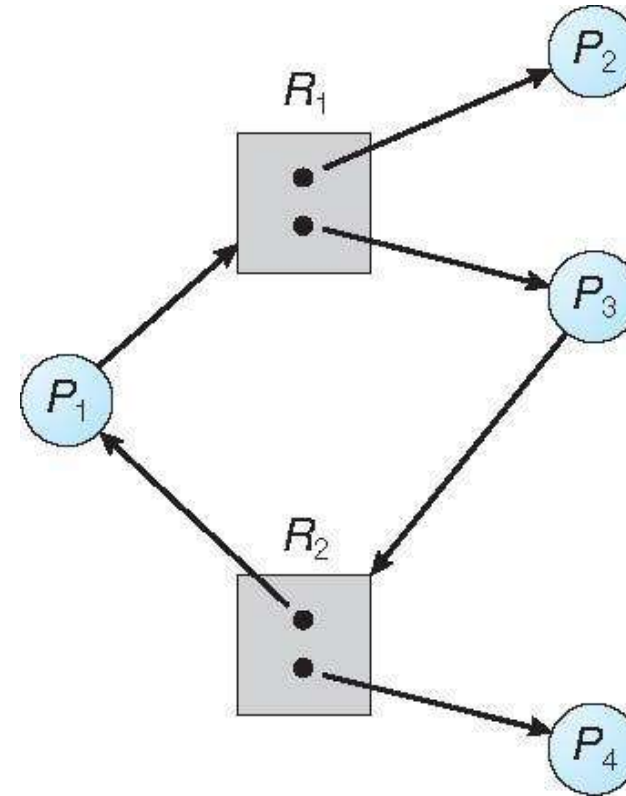
OPERATING SYSTEMS

Resource-Allocation Graph



$P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

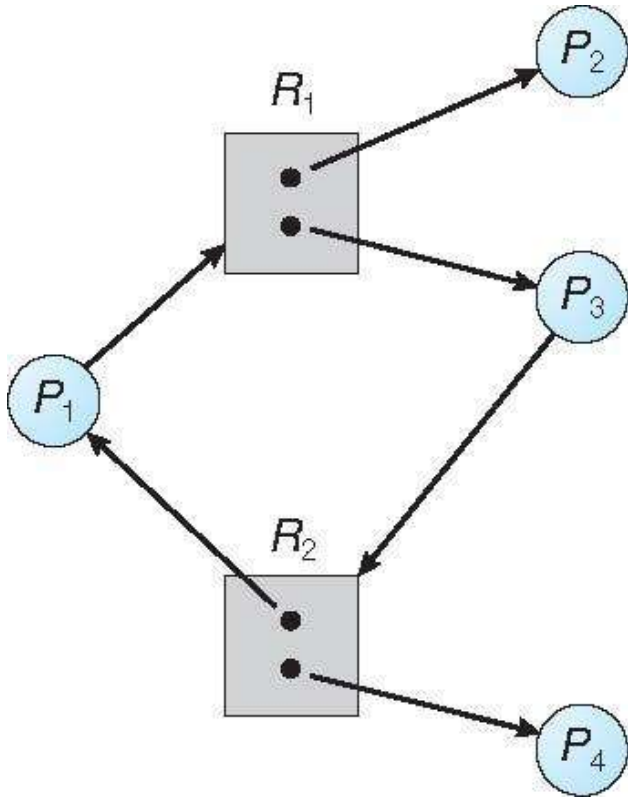
$P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$



$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

OPERATING SYSTEMS

Resource-Allocation Graph



$P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$

OPERATING SYSTEMS

Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, the system may or may not be in a deadlocked state



- Generally speaking there are three ways of handling deadlocks:
 - Deadlock prevention or avoidance - Do not allow the system to get into a deadlocked state.
 - Allow the system to enter into deadlocked state, detect it, and recover.
 - Ignore the problem all together and pretend that deadlocks never occur in the system.

4 Necessary conditions for deadlock to occur

- **Mutual Exclusion**

- At least one resource must be non sharable
 - 4 Ex. printers and tape drives, mutex locks
- Sharable resources do not require mutual exclusion
 - 4 Ex . Read-only files

- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources

- Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.
- Low resource utilization; starvation possible

- **No Preemption –**
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
 - Preempted resources are added to the list of resources for which the process is waiting
 - Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting
- **Circular Wait –**
 - Each resource will be assigned with a numerical number.
 - A process can request the resources increasing/decreasing order of numbering.

OPERATING SYSTEMS

Deadlock Detection

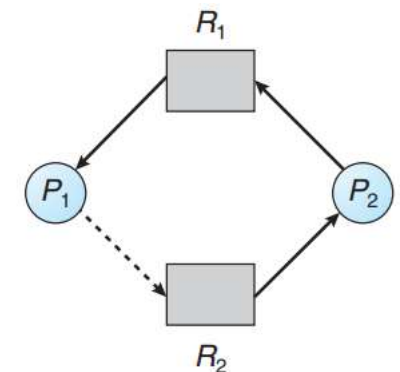
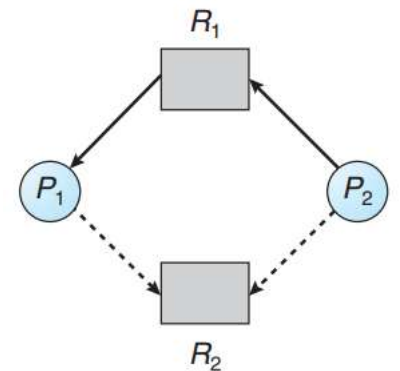
- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme



OPERATING SYSTEMS

Single Instance of Each Resource Type

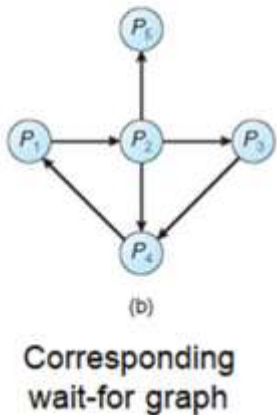
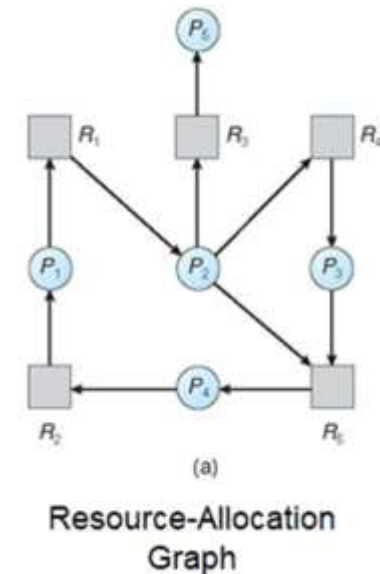
- In a resource allocation graph, a claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge is represented in the graph by a dashed line.
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph



OPERATING SYSTEMS

Resource-Allocation Graph and Wait-for Graph

- If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph.
- We obtain wait-for graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.
- An edge from P_i to P_j implies that process P_i is waiting for process P_j to release a resource that P_i needs.
- An edge $P_i \rightarrow P_j$ exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for some resource R_q .
- A **deadlock exists** in the system if and only if the **wait-for graph contains a cycle**.
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.



- **Available:** A vector of length m indicates the number of available resources of each type. If $\text{Available}[j] = k$, then k instances of resource type R_j are available
- **Max:** An $n \times m$ matrix defines the maximum demand of each process. If $\text{Max}[i][j] = k$, then process P_i may request at most k instances of resource type R_j .
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i][j] = k$, then process P_i is currently allocated k instances of resource type R_j
- **Request:** An $n \times m$ matrix indicates the current request of each process. If ***Request*** $[i][j] = k$, then process P_i is requesting k more instances of resource type R_j .

OPERATING SYSTEMS

Detection Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:
 - (a) ***Work = Available***
 - (b) For ***i = 1, 2, ..., n***, if ***Allocation_i ≠ 0***, then ***Finish[i] = false***; otherwise, ***Finish[i] = true***

 2. Find an index ***i*** such that both:
 - (a) ***Finish[i] == false***
 - (b) ***Request_i ≤ Work***
- If no such ***i*** exists, go to step 4



3. **$Work = Work + Allocation_i$**
 $Finish[i] = true$
go to step 2
4. If **$Finish[i] == false$** , for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if **$Finish[i] == false$** , then P_i is deadlocked. If **$Finish[i] == true$** for all i , then the system is in a safe state

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state

OPERATING SYSTEMS

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$

OPERATING SYSTEMS

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>			<u>Request</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	1			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$

OPERATING SYSTEMS

Example of Detection Algorithm (Cont.)

- P_2 requests an additional instance of type **C**

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

OPERATING SYSTEMS

Deadlock Avoidance



- Deadlock-prevention algorithms, prevent deadlocks by limiting how requests can be made.
- The limits ensure that at least one of the necessary conditions for deadlock cannot occur.
- Possible side effects of preventing deadlocks by this method, however, are low device utilization and reduced system throughput.
- **Deadlock avoidance** requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
- With this additional knowledge of complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock
- To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process

OPERATING SYSTEMS

Deadlock Avoidance

- The simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.
- A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist.
- The resource allocation ***state*** is defined by the number of available and allocated resources and the maximum demands of the processes.



OPERATING SYSTEMS

Safe State



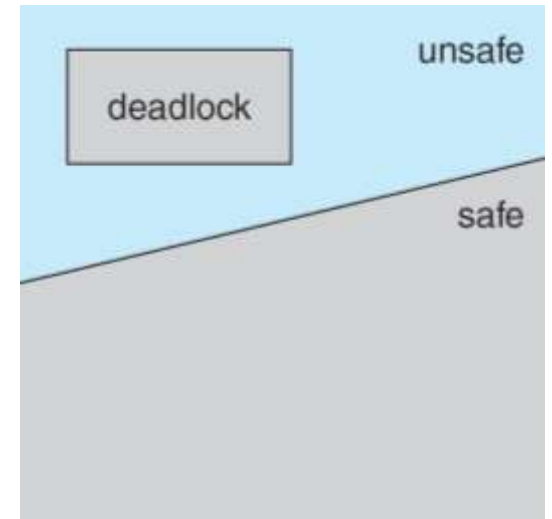
- A state is **safe** if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a **safe sequence**.
- A sequence of processes $\langle P_1, P_2, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$.
- In this situation, if the resources that P_i needs are not immediately available, then P_i can wait until all P_j have finished.
- When they have finished, P_i can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate.
- When P_i terminates, P_{i+1} can obtain its needed resources, and so on. If no such sequence exists, then the system state is said to be **unsafe**

OPERATING SYSTEMS

Safe, Unsafe and Deadlock States



- A safe state is not a deadlocked state.
- Conversely, a deadlock state is an unsafe state.
- Not all unsafe states are deadlocks.
- An unsafe state may lead to a deadlock.
- As long as the state is safe, the OS can avoid unsafe states.
- In an unsafe state, the OS cannot prevent processes from requesting resources in such a way that a deadlock occurs.
- The behavior of processes controls unsafe states.
- If a system is in safe state → no deadlocks
- If a system is in unsafe state → possibility of deadlock
- Goal for Avoidance → ensure that a system will never enter an unsafe state.



OPERATING SYSTEMS

Deadlock avoidance algorithms

- Avoidance algorithms ensure that the system will never deadlock.
- The idea is simply to ensure that the system will always remain in a safe state.
- Initially, the system is in a safe state. Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or whether the process must wait.
- The request is granted only if the allocation leaves the system in a safe state.
- In this scheme, if a process requests a resource that is currently available, it may still have to wait.
- Thus, resource utilization may not be optimal

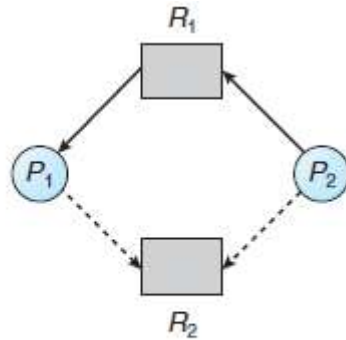
OPERATING SYSTEMS

Resource Allocation Graph algorithm

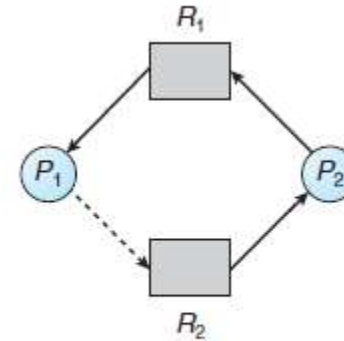
- In addition to the request and assignment edges in a resource allocation graph, we introduce a new type of edge, called a **claim edge**
- A claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge resembles a request edge in direction but is represented in the graph by a dashed line.
- When process P_i requests resource R_j , the claim edge $P_i \rightarrow R_j$ is converted to a request edge. Similarly, when a resource R_j is released by P_i , the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$
- The resources must be claimed a priori in the system. That is, before process P_i starts executing, all its claim edges must already appear in the resource-allocation graph.
- Now suppose that process P_i requests resource R_j . The request can be granted only if converting the request edge $P_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow P_i$ does not result in the formation of a cycle in the resource-allocation graph.

OPERATING SYSTEMS

Resource Allocation Graph algorithm example



Resource-allocation graph for deadlock avoidance.



An unsafe state in a resource-allocation graph.

- Consider the above resource-allocation graph.
- Suppose that P_2 requests R_2 . Although R_2 is currently free, we cannot allocate it to P_2 , since this action will create a cycle in the graph below.
- A cycle indicates that the system is in an unsafe state.
- If P_1 requests R_2 , then a deadlock will occur.

OPERATING SYSTEMS

Banker's algorithm

- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.
- Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system.
- We need the following data structures, where n is the number of processes in the system and m is the number of resource types



OPERATING SYSTEMS

Banker's algorithm data structures



- **Available** – A vector of length m indicates the number of available resources of each type. If **Available** $[j]$ equals k , then k instances of resource type R_j are available.
- **Max.** - An $n \times m$ matrix defines the maximum demand of each process. If **Max** $[i][j]$ equals k , then process P_i may request at most k instances of resource type R_j .
- **Allocation** - An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If **Allocation** $[i][j]$ equals k , then process P_i is currently allocated k instances of resource type R_j .
- **Need** - An $n \times m$ matrix indicates the remaining resource need of each process. If **Need** $[i][j]$ equals k , then process P_i may need k more instances of resource type R_j to complete its task.
- **Need** $[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$.
- Treat each row in the matrices **Allocation** and **Need** as vectors. The vector **Allocation** $_i$ specifies the resources currently allocated to process P_i ; the vector **Need** $_i$ specifies the additional resources that process P_i may still request to complete its task.
- These data structures vary over time in both size and value.

OPERATING SYSTEMS

Banker's algorithm for determining whether or not a system is in a safe state



1. Let **Work** and **Finish** be vectors of length m and n , respectively.

Initialize **Work** = **Available** and **Finish**[i] = **false** for $i = 0, 1, \dots, n - 1$.

2. Find an index i such that both

a. **Finish**[i] == **false**

b. **Need** $_i \leq \mathbf{Work}$

If no such i exists, go to step 4

3. **Work** = **Work** + **Allocation** $_i$

Finish[i] = **true**

Go to step 2.

4. If **Finish**[i] == **true** for all i , then the system is in a safe state.

- This algorithm may require an order of $m \times n^2$ operations to determine whether a state is safe.

OPERATING SYSTEMS

Banker's algorithm for determining whether requests can be safely granted



Let ***Request_i*** be the request vector for process *P_i* . If ***Request_i*** [*j*] == *k*, then process *P_i* wants *k* instances of resource type *R_j* .

When a request for resources is made by process *P_i* , the following actions are taken:

1. If ***Request_i*** ≤ ***Need_i*** , go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
2. If ***Request_i*** ≤ ***Available***, go to step 3. Otherwise, *P_i* must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to process *P_i* by modifying the state as follows:

$$\mathbf{Available} = \mathbf{Available} - \mathbf{Request}_i ;$$

$$\mathbf{Allocation}_i = \mathbf{Allocation}_i + \mathbf{Request}_i ;$$

$$\mathbf{Need}_i = \mathbf{Need}_i - \mathbf{Request}_i ;$$

If the resulting resource-allocation state is safe, the transaction is completed, and process *P_i* is allocated its resources.

If the new state is unsafe, then *P_i* must wait for ***Request_i*** , and the old resource-allocation state is restored.

- Consider a system with five processes P_0 through P_4 and three resource types A , B , and C .
- Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances.

- Suppose that, at time T_0 , the following snapshot of the system has been taken:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies the safety requirement. Hence, we can immediately grant the request of process P_1 .
- A request for (3,3,0) by P_4 cannot be granted, since the resources are not available.
- A request for (0,2,0) by P_0 cannot be granted, even though the resources are available, since the resulting state is unsafe.

- Consider a system with five processes P_0 through P_4 and three resource types A , B , and C .
- Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances.
- Suppose that, at time T_0 , the following snapshot of the system has been taken:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies the safety requirement. Hence, we can immediately grant the request of process P_1 .
- A request for $(3, 3, 0)$ by P_4 cannot be granted, since the resources are not available.
- A request for $(0, 2, 0)$ by P_0 cannot be granted, even though the resources are available, since the resulting state is unsafe.



THANK YOU

Pavan A C

Department of Computer Science and Engineering

pavanac@pes.edu